SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 1 – CO1

Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Student Name	**P. Lakshmi Gayathri Navaneetha**	**Register Number**	**192321169**
Department / Section	**Information Technology**	**Academic Year / Semester**	**4 year**
Faculty Name	**DR.Kesavan R**	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5		
2	Evaluate the Role of Process in Software Quality	2		
3	Evaluate Testing as a Process	6		
4	Apply and Critique Basic Definitions	4		

5	Evaluate the Application of Testing Principles	3		
6	Evaluate the Tester's Role	3		
7	Trace the Origins of Defects	4		

8	Evaluate Defect Classes and Severity	4		
9	Design and Appraise a Defect Repository	3		
10	Evaluate Defect Examples and Prevention Strategies	4		
11	Recommend and Defend a Quality Improvement Plan	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of the Original Plan	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence Based Defence	2		
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.		
2	A well-justified position on how process affects quality, backed by data from the case.		
3	A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.		
4	Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.		
5	A justified assessment of the developers' assertion using testing principles tied to evidence from the case.		
6	A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.		
7	A justified origin for every incident and an interpretation of the overall pattern of defect origins.		
8	A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.		

9	A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.		
10	A root-cause analysis of three defects with justified prevention strategies that take cost into account.		
11	A technically justified quality improvement plan backed by the evidence built up in earlier tasks.		
12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence		

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	Status	Observation
Defect data calculations (cost per phase, totals)		
Phase-containment / detection figures		

Origin assigned to every incident (I1– I10)		
Class, severity and priority justified		
Defect repository fields with reasons		
Root cause → prevention strategy link		
Cost–benefit of prevention strategies		
Interpretation of the defect pattern		
Figures and conclusions consistent		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Defect cost calculations shown step by step	☒ Yes ☐ Partly ☐ No
All ten incidents classified (origin, class, severity, priority)	☒ Yes ☐ Partly ☐ No
Error, fault and failure separated for I3, I5 and I6	☒ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☒ Yes ☐ Partly ☐ No
Repository design justified field by field	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING (CSA37)
ASSIGNMENT 1 – CO1

CO1 Coverage: Testing as an engineering activity, role of process in software quality, testing as a process, basic definitions, software testing principles, tester's role, origins of defects, defect classes, defect repository, defect examples.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.

ID	Incident
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

Task 1 – Evaluate Testing as an Engineering Activity **5 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)
- Which of these characteristics were missing in the MediQueue project, with evidence from the case
- How the absence of each characteristic contributed to the incidents reported
- What an engineered testing approach would have looked like for this project

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected

A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

Ans:

Was MediQueue's Testing an Engineering Activity?

Engineering means achieving a required result on purpose, through planning, measurement, agreed standards and work that can be repeated. To decide whether MediQueue's team engineered quality or merely hoped for it, I compare their actual practice with these four characteristics, and with two supporting conditions (independent roles, and tools). Everything I rely on comes from the facts given in the case.

My verdict: it was not an engineering activity. Five developers spent eight months building the product, checked their own code shortly before release, and management signed it off because a demonstration showed no errors. A demonstration only proves that the paths the presenter chose to show are working; it says nothing about the paths nobody tried. The defect figures settle the matter: patients, not the team, found 60 of the 100 defects.

Table 1: MediQueue's testing judged against the characteristics of an engineering activity

Characteristic	What engineering practice expects	What the case shows	Incidents that followed
Planning	Test goals, levels, schedule, resources, risks and entry/exit criteria written down before testing starts	No test plan exists. Testing was done "shortly before release", after eight months of coding, with no time or staff reserved for it	I1, I6, I9, I10: concurrency, interface and load behaviour were never planned for
Measurement	Defects per phase, coverage and cost are tracked and used to decide whether to release	The release criterion was "the demo showed no errors". Only 40 of 100 defects were found before release, and nothing flagged this as unacceptable	All ten; this is why 60% of defects escaped

Characteristic	What engineering practice expects	What the case shows	Incidents that followed
Standards and defined process	Requirement and design reviews, a standard test-case format, traceability to requirements	Requirements and design had "no formal review"; coding had a "developer self-check only"; the Sunday and GST rules were never documented	I2, I3, I7
Repeatability	Written test cases, data and expected results that anyone can re-run after each change	A demo cannot be re-run. The one recorded test that surfaces in the incidents (refund) was passed against a wrong expected value	I4, I5, I8
Independence and roles	Someone whose job is to find defects, with testing skills	There was no tester. Developers checked their own code, so the same assumptions went into both the code and the "test"	I5, I6
Tools and environment	Test bed, test data, defect tracker, automation for repeat runs	Complaints sat in e-mails and a spreadsheet; no SMS sandbox or load environment existed	I9, I10

How each missing characteristic produced incidents

- **No planning:** nobody asked "what happens on a slow network, when two users act in the same second, or when the SMS key changes?" I1, I6, I9 and I10 are exactly the defects that appear only when such tests are planned.
- **No measurement:** without an exit criterion the team could not see its own risk. Of the 52 requirement and design defects, only 5 were found before release (2 + 3), and the 20 found in testing were just 26.7% of the 75 defects then present (20 ÷ 75).
- **No standards or reviews:** a missing Sunday rule (I2) and a misread GST rule (I3) can only be caught by checking the requirement with the people who own the rule. The wrong menu name in the manual (I7) is the same gap applied to a document.
- **No repeatability:** with no written cases or expected values, boundary inputs such as a 41-character name (I4) or a patient aged exactly 60 (I5) are tested only by luck. The one case that was written (refund) used a wrong expected value that nobody rechecked (I8).
- **No independence:** a developer who believes that two doctors never save together (I6) will never write the test that proves otherwise.

What an engineered approach would have looked like

- A short test policy and a test plan for every release covering goals, levels (unit, integration, system, acceptance), schedule and risks.
- Written exit criteria, for example: all planned high-risk tests run, no open Critical or High defect, at least 95% of planned tests passed, hospital sign-off.
- Reviews of the requirements with billing and OPD staff, and of the design (concurrency, shared data, interfaces), before coding starts.
- Test cases derived by equivalence classes and boundary values, each with an expected result taken from the requirement and traced back to it.
- System tests for concurrency, performance on the hospital Wi-Fi and the SMS interface, run in a proper test environment.
- A defect repository with simple measures (defects per phase, cost, escapes) reviewed

at every release.

Conclusion: in MediQueue, testing was an event, not an engineering process. Every missing characteristic can be linked to at least one incident, so the incidents were not bad luck; they were the predictable result of how the work was organised.

Task 2 – Evaluate the Role of Process in Software Quality **2 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- The role of a defined process in achieving software quality
- Whether better developers alone would have prevented the incidents
- How a process maturity view (for example, TMM levels) would place the current team

Your position must be defended with evidence from the case.

Evidence Expected

A justified position on the role of process in quality, supported by the case data.

Ans:

Good People or a Good Process?

My position is that MediQueue's problem is mainly one of process. Better developers would help, but they would not have stopped most of these incidents, because most defects were born before coding, or outside the code altogether.

What a defined process contributes. A process makes quality independent of any one person. Reviews catch mistakes before they become code, a test plan decides in advance what must be checked, exit criteria turn "we think it is fine" into a measurable decision, and a defect repository lets the team learn. It also catches the mistakes that skilled people make, because everyone is fallible.

Table 2: Would a better developer on their own have prevented the incident?

Incident	Prevented by skill alone?	Reason
I1, I6	Possibly	An experienced developer may think of concurrency, but a design review and a concurrent-use test are still needed to be sure
I4, I5	Probably	Input-length and age-boundary handling is a coding skill, yet a boundary test would catch both even for an average developer
I10	Partly	Also needs an agreed performance target (a requirement) and a load test
I2, I3	No	The Sunday rule was never documented and the GST rule was misread; coding skill cannot supply a rule the developer was never given
I7	No	A documentation defect; it needs a document review

Incident	Prevented by skill alone?	Reason
I8	No	A fault in the test itself; it needs a second-person review of test cases and an independent tester
I9	No	The provider changed its key outside the team's control; it needs interface monitoring and configuration control

At best, skill alone might have prevented 4 or 5 of the 10 incidents. The defect data agree: only 38 of the 100 defects were introduced in coding, while 52 (30 + 22) came from requirements and design. Even if better developers halved coding defects, only 19 would disappear and 81 would remain, so "better developers" addresses less than a fifth of the problem.

Where the team stands on a maturity scale

On the Testing Maturity Model (TMM) I place the team at **Level 1 (Initial)**. Testing is not separated from debugging, its only aim is to show that the software runs (the demo), and there is neither a test plan nor a trained tester. The next target is **Level 2 (Phase Definition)**, whose goals match exactly what is missing: separate testing from debugging, start test planning, and adopt basic testing techniques. **Level 3 (Integration)** would follow, with a test group, a test life cycle built into the development life cycle, and training.

Task 3 – Evaluate Testing as a Process **6 Marks**

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- Testing as a process with planning, design, execution, evaluation and reporting activities
- The relationship between verification and validation activities and each development phase
- The phase in which each type of MediQueue defect should have been detected
- Using the defect data table, the cost the hospital paid because defects were found late
- The likely saving if requirements and design reviews had been conducted
- The limits of your estimate and the assumptions it depends on

You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.

Evidence Expected

A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

Ans:

Testing as a Process Across the Life Cycle

MediQueue treated testing as one event before release. In this answer I (1) describe the test process the project should have followed, (2) relate verification and validation to each phase, (3) name the phase in which each incident should have been caught, (4) cost the late discovery using the defect table, and (5) estimate the saving from reviews, with (6) the limits of that estimate.

1. Testing viewed as a process

Table 3: Test process activities applied to MediQueue

Activity	What it would involve in MediQueue	Output
Planning	Set test goals, levels, risks (concurrency, money, privacy), schedule and exit criteria	Test plan
Design	Derive cases from requirements: age 59/60/61, name length 40/41/42, a Sunday booking, displayed fee = charged fee, two bookings in the same second	Test cases with expected results
Set-up	Build a test bed: test doctors and patients, an SMS sandbox, hospital-like Wi-Fi	Test environment
Execution	Run the planned tests at unit, integration and system level; re-run after every fix	Test log
Evaluation	Compare actual with expected results (the oracle), decide pass or fail, check against exit criteria	Verdicts and defect reports
Reporting and closure	Record defects, a test summary and lessons learnt in the repository	Summary report, repository entries

2. Verification and validation in each phase

Table 4: Verification and validation by development phase

Phase	Verification (building it right?)	Validation (building the right thing?)	Defects targeted
Requirements	Checklist review of requirements; testability check	Acceptance tests planned with hospital staff	I2, I3, I10
Design	Design inspection of concurrency, data handling, interfaces	System test cases designed	I1, I6, I9
Coding	Code review, coding standard, boundary-value unit tests	–	I4, I5
Integration and system test	Interface, regression and test-case review	Performance and concurrency tests; user acceptance test	I1, I3, I6, I9, I10
Documentation	Check manual against the running system	A real user follows the manual	I7
Operation	Interface monitoring, regression after changes	Patient feedback	I9

3. Where each incident should have been detected

Table 5: Intended detection phase of each incident

Incident	Defect	Should be caught in	Activity that would reveal it
I1	Same token issued twice	Design	Design inspection (concurrency), then a concurrent-booking system test
I2	Sunday rule missing	Requirements	Requirement review with OPD staff
I3	₹500 shown, ₹550 charged	Requirements	Review with billing; system test comparing displayed and charged fee
I4	Crash for names over 40 characters	Coding	Unit test on boundary lengths (40, 41)
I5	No senior priority at age 60	Coding	Boundary test (59, 60, 61); code review
I6	Wrong patient on PDF	Design	Design inspection (shared data); concurrent-save system test
I7	Manual names a missing menu	Documentation	Review the manual against the application
I8	Wrong expected value in refund test	Testing	Second-person review of the test case
I9	SMS stops after key change	Design / integration	Interface test, configuration control, production monitoring
I10	Schedule screen takes 25 s	Requirements / system test	A performance target, then load testing on hospital Wi-Fi

Under this plan five incidents (I1, I2, I3, I6, I10) belong to requirements and design, two (I4, I5) to coding and three (I7, I8, I9) to documentation, testing and operation. In the real project, patients found all ten.

4. What late detection cost the hospital

Table 6: Cost of the last release's 100 defects, using the case's cost per defect

Phase of detection	Defects detected	Cost per defect (₹)	Total cost (₹)	Share of total
Requirements	2	500	1,000	0.04%
Design	3	1,500	4,500	0.2%
Coding	15	3,000	45,000	1.7%
Testing	20	8,000	1,60,000	6.1%
Production (after release)	60	40,000	24,00,000	91.9%
Total	**100**	**avg 26,105**	**26,10,500**	**100%**

Calculation. Production: 60 × 40,000 = 24,00,000. Testing: 20 × 8,000 = 1,60,000. Coding: 15 × 3,000 = 45,000. Design: 3 × 1,500 = 4,500. Requirements: 2 × 500 = 1,000. Sum = ₹26,10,500,

an average of ₹26,105 per defect.

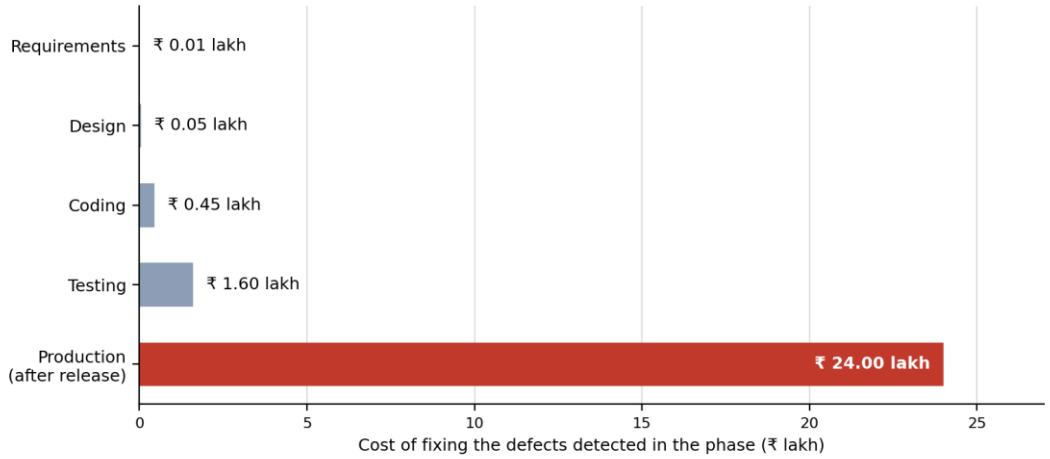


Figure 1: Defect cost by the phase in which the defects were detected

What it means. Patients found 60% of the defects, and those defects produced 91.9% of the cost. One production defect costs 80 times a requirements defect (40,000 ÷ 500) and 5 times a defect found in testing (40,000 ÷ 8,000). If the 60 production defects had been caught in testing instead, they would have cost 60 × 8,000 = ₹4,80,000 rather than ₹24,00,000, a difference of ₹19,20,000; this is an upper limit, since testing would not have caught every one. If every defect had been fixed in the phase that introduced it, the 95 defects that have a unit cost would have cost 30 × 500 + 22 × 1,500 + 38 × 3,000 + 5 × 8,000 = ₹2,02,000. Against the actual ₹26,10,500, late detection added about ₹24 lakh (26,10,500 − 2,02,000 = 24,08,500), roughly 13 times the ideal cost.

How the defects flowed through the phases. Table 7 follows the 100 defects phase by phase. "Present" means defects carried over plus those newly introduced; "detection efficiency" is detected ÷ present.

Table 7: Defect flow and detection efficiency of the last release

Phase	Introduced	Present in phase	Detected	Detection efficiency	Passed on
Requirements	30	30	2	6.7%	28
Design	22	50	3	6.0%	47
Coding	38	85	15	17.6%	70
Testing	5	75	20	26.7%	55
Documentation	5	60	0	0.0%	60
Production	–	60	60	100%	0

Requirements and design, where 52 defects were introduced, caught only 5 (9.6%). Only 40 of the 100 defects (40%) were found before release, so the "no errors in the demo" decision was taken while 60 defects were still hidden in the product.

5. Probable saving from requirements and design reviews

I model the reviews with the flow in Table 7. A requirements review and a design review each

remove a fraction r of the defects present in their phase (instead of today's 6.7% and 6.0%); coding and testing keep their present rates (17.6% and 26.7%). Defects removed in a phase are charged at that phase's cost and the rest pass on. I tried r = 40%, 60% and 80%, and assumed ₹1,00,000 as the cost of running both reviews (about 125 person-hours at ₹800 per hour, including preparation).

Table 8: Estimated saving from requirements and design reviews

Review efficiency	Defects found by reviews	Defects reaching patients	Total cost after reviews (₹)	Gross saving (₹)	Review cost (₹)	Net saving (₹)	Net saving %
40%	28.0	46.1	20,26,808	5,83,692	1,00,000	4,83,692	18.5%
60%	38.4	39.8	17,61,399	8,49,101	1,00,000	7,49,101	28.7%
80%	46.4	35.0	15,55,854	10,54,646	1,00,000	9,54,646	36.6%

At a realistic 60% review efficiency, defects reaching patients fall from 60 to about 40, the total cost falls from ₹26,10,500 to about ₹17,61,000, and the net saving after paying for the reviews is about ₹7.5 lakh (28.7%). Reviews still pay at 40% efficiency (net ₹4.8 lakh), because ₹1 lakh spent on reviews avoids many ₹40,000 production fixes. In the same model, a testing phase that catches 60% of the defects present (instead of 26.7%) would cut production defects from 60 to 35 and save about ₹8 lakh, which shows that reviews and better testing complement each other.

6. Limits of the estimate and its assumptions

- **Defect flow:** the case does not say where each detected defect came from, so I assumed that defects missed in one phase pass to the next. The real flow may differ.
- **Review efficiency:** 40–80% is an assumption because the team has no review data; poorly run reviews would sit at the low end.
- **Unit costs:** I used the per-defect costs in the case, which cover rework only. Patient harm, lost trust, legal exposure and hospital staff time are excluded, so the true cost of production defects is higher.
- **Documentation defects:** the case gives no cost for them; I treated the five that remained as production defects.
- **One release:** the data come from one release of one project with 100 defects, and defects never discovered are not counted.
- **Review cost:** ₹1,00,000 is my own estimate; a much higher cost would reduce the benefit but not remove it.

Task 4 – Apply and Critique Basic Definitions — 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- The meaning of error, fault (defect), failure, test case, test oracle and test bed in the

MediQueue context

- For incidents I3, I5 and I6, the error, the fault and the failure
- Why the confusion in terminology made incident I8 possible
- Whether the team needs a common glossary, and what it must contain

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected

Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

Ans:

Applying and Critiquing the Basic Definitions

Terminology is more than words. If "bug", "error" and "failure" all mean the same thing, the team cannot tell whether a problem lies with a person, in the product, or in what the user saw, and so cannot trace or prevent its cause. I first define each term for MediQueue and then apply them to the incidents.

Table 9: Basic definitions in the context of MediQueue

Term	Meaning	MediQueue example
Error	A mistake made by a person	A developer assumes that only one doctor saves a prescription at a time
Fault (defect, bug)	The flaw in the software or a document that the error leaves behind	The PDF generator keeps patient data in a variable shared by all doctors
Failure	The wrong behaviour a user sees when the faulty code runs	A prescription PDF shows the previous patient's details
Test case	Inputs, conditions and expected result for one check	Book a Sunday appointment → the system must refuse it and show "OPD closed"
Test oracle	The source that tells what the correct result is (requirement, rule, calculation, expert)	The approved fee table (₹500 + tax) decides whether ₹550 is right
Test bed	The environment in which tests run: hardware, network, data, tools, stubs	A copy of the app with test doctors and patients, an SMS sandbox and hospital Wi-Fi conditions

Table 10: Error, fault and failure for incidents I3, I5 and I6

Incident	Error (human mistake)	Fault (flaw in the product)	Failure (what the user saw)
I3	The analyst or developer misread the GST rule and applied it in two different ways	Two separate fee calculations: the booking page shows the base fee while the payment module adds tax	Page shows ₹500; patient pays ₹550

Incident	Error (human mistake)	Fault (flaw in the product)	Failure (what the user saw)
I5	The developer read "senior citizen" as "over 60", or compared the age on the wrong date	A condition such as age > 60 (or age taken on the booking date, not the appointment date)	A patient turning 60 on the appointment day gets no priority
I6	The developer assumed that two doctors never save at the same moment	Patient details held in shared data that the second save overwrites before the PDF is built	The PDF shows the previous patient's details

How the confusion allowed I8

In I8 a refund test was recorded as "Pass" because the output was compared with an incorrect expected value. The tester made an **error**, which put a **fault** into the test case (a wrong oracle), and the **failure** was a false verdict. A team that calls everything a "bug" does not separate a fault in the product from a fault in the test, so no one asks "where did this expected value come from?". No test oracle had been defined, the value was written from memory rather than from the requirement, and the "Pass" was trusted. The refund defect stayed hidden, so a loose word concealed a wrong test.

Does loose terminology really damage quality?

Yes, in four ways:

- **Root-cause analysis breaks down:** a failure is caused by a fault, and a fault by an error, and each needs a different remedy (a code fix, a review checklist, training).
- **Measurement is distorted:** the figure of 100 defects mixes failures reported by patients with faults in the code, so one fault can be counted many times.
- **Communication fails:** telling management "there is a bug" says nothing about severity or cause.
- **Fix verification is weak:** nobody records which test case and oracle prove that a fault has really been removed.

A common glossary

The team needs a one-page glossary agreed by developers, testers and hospital staff and kept next to the defect repository.

Table 11: Minimum contents of the MediQueue testing glossary

Group	Terms to define	Why it is needed
Defect terms	Error, fault/defect, failure, incident	Separate cause, flaw and symptom in every report
Test terms	Test case, expected result, test oracle, test bed, test log, regression test	Make tests repeatable and avoid wrong expected values (I8)
Rating terms	Severity and priority levels with MediQueue examples	End disputes about what to fix first
Defect states	New, assigned, fixed, verified, closed, reopened	Follow each defect through to closure

Group	Terms to define	Why it is needed
Product terms	Build, release, hotfix, appointment day, senior citizen, fee, token	Remove ambiguity such as "senior at 60" (I5)

Task 5 – Evaluate the Application of Testing Principles **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct".

Which testing principles prove this claim wrong?

Discuss:

- At least four software testing principles that apply to the MediQueue situation
- For each principle, the incident or decision in the case that violates or confirms it
- Which principle, if followed, would have had the largest effect on quality, and why

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected

A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

Ans:

Which Testing Principles Show the Developers' Claim Is Wrong?

The claim contains two mistakes: it treats "no failures in the tests that were run" as proof of correctness, and it assumes that everything could be tested. I apply eight principles and tie each one to evidence from the case.

Table 12: Testing principles applied to MediQueue

Principle	Followed or broken	Evidence from the case
Testing shows the presence of defects, never their absence	Broken	A demo with no visible errors was accepted as proof of quality; 60 defects were still hidden and patients found them
Exhaustive testing is impossible, so tests must be chosen by risk	Broken	Names, dates, ages, fees, networks and simultaneous users give far more combinations than any team can run. "We tested everything" is impossible, and no risk-based selection was made
Testing must be planned	Broken	No test plan; testing began just before release, so no time went to load, concurrency or SMS tests (I1, I6, I9, I10)
Testing belongs in the whole life cycle, starting early	Broken	52 of 100 defects arose in requirements and design but only 5 were caught there; the cost rose from ₹500 to ₹40,000 per defect. Late detection cost ₹26.1 lakh, 91.9% of it in production
Every test case must state its expected result	Broken	The refund test used a wrong expected value and was passed (I8)

Principle	Followed or broken	Evidence from the case
Test valid as well as invalid and unexpected inputs	Broken	A 41-character name (I4), a Sunday booking (I2) and the age-60 boundary (I5) are extreme or invalid inputs that the demo never used
Testing should be done by an independent person	Broken	Developers tested their own code, so the same assumption (single user, "over 60") lived in both code and test (I5, I6)
Defects cluster: where one is found, more are nearby	Confirmed	I1 and I6 both come from simultaneous use of shared data; once one was found the other should have been searched for

The principle with the greatest impact

I judge that **"testing belongs in the whole life cycle and starts early"** would have changed the most. Half the incidents (I1, I2, I3, I6, I10) begin in requirements and design, and those phases caught only 9.6% of their own defects, while the cost per defect grows from ₹500 in requirements to ₹40,000 in production. Planning and independence matter, but both work far better when they start at the requirements stage. Writing test cases from the requirements would have exposed the missing Sunday rule and the fee mismatch before a line of code existed.

Task 6 – Evaluate the Tester's Role **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

- The role and responsibilities of a tester across the life cycle
- The advantages and limitations of developers testing their own code
- Which incidents a tester would most likely have caught, and at which stage
- The qualities a tester for a healthcare application must have

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

Ans:

Would an Independent Tester Have Made a Difference?

My position: an independent tester would have changed the outcome considerably, but would not have removed every defect. Working across the life cycle, a tester would very probably have caught seven of the ten incidents; the other three (I1, I6, I9) also need design review, specialised tests and production monitoring.

The tester's role through the life cycle

- **Requirements:** review for missing, vague or untestable statements, asking "which days is the OPD open? is the fee shown with tax? how fast must the schedule load?"
- **Design and planning:** write the test plan, risks and exit criteria; design test cases and the test bed.
- **Coding and integration:** support unit testing, run integration tests and maintain the regression suite.
- **System and acceptance:** run functional, load, concurrency and security tests and support user acceptance.
- **Throughout:** report defects with reproduction steps, verify fixes, keep the repository and give management evidence for the release decision.

Developers testing their own code

Advantages: the developer knows the code, finds simple faults quickly and fixes them cheaply, so developer unit testing is valuable and should continue. **Limitations:** the developer tests what he or she believes the code does, so a wrong assumption is carried into the test; there is a natural wish to show the work is good; testing is squeezed to the end by deadlines; and nobody checks how the requirement was interpreted. MediQueue shows this: I5 (age-60 boundary) and I6 (single-user assumption) both came from the developers' own assumptions.

Table 13: Incidents a tester would most probably have caught

Incident	Caught by a tester?	Stage and method
I2	Yes	Requirement review and test design: ask which days the OPD works
I3	Yes	Requirement review; system test comparing the fee displayed with the amount charged
I4	Yes	Unit or system test with boundary-length names (40 and 41 characters)
I5	Yes	Test design with boundary ages 59, 60 and 61 on the appointment day
I7	Yes	Document review: follow the manual step by step on the real application
I8	Yes	Second-person review of the test case; trace the expected value to the requirement
I10	Yes	System load test on hospital Wi-Fi, once a performance target is agreed
I1	Maybe	Only if a concurrency test is designed (needs tools and skill); design inspection is the stronger safeguard
I6	Maybe	As for I1: needs a concurrent-save test and a design review of shared data
I9	Partly	Interface test against an SMS sandbox; the later key change needs monitoring and configuration control

That makes 7 "yes", 2 "maybe" and 1 "partly". In terms of the defect data, if a tester lifted detection in the testing phase from 26.7% to 60% of the defects present (Task 3), production defects would fall from 60 to about 35 and about ₹8 lakh would be saved. That is large, but 35 defects would still reach patients, so a tester alone is not enough, which is why my plan also includes reviews.

Qualities needed in a healthcare tester

- **Domain knowledge:** outpatient workflow, fees and taxes, senior-citizen rules and prescriptions.
- **Eye for detail and curiosity:** notices a ₹50 difference, a wrong menu name or a wrong expected value.
- **Sensitivity to patient safety and privacy:** knows that showing one patient's data to another (I6) is a serious breach.
- **Technical skill:** test design techniques, concurrency and load testing, defect tracking and basic automation.
- **Communication and ethics:** reports defects clearly and without blame, keeps health data confidential and can say "not ready to release".

Task 7 – Trace the Origins of Defects **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- The origin of each incident: requirements, design, code, testing, documentation or environment
- The evidence in the case that supports each origin you assign
- Incidents that may have more than one origin, and how you decided
- What the overall pattern of origins tells the hospital about where to invest effort

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

Ans:

Where Do MediQueue's Defects Come From?

For each incident I give one main origin, the supporting evidence, and any second possible origin with the reason I preferred the main one.

Table 14: Origin of each incident

Incident	Main origin	Evidence from the case	Possible second origin and why I chose the main one
I1	Design	Duplicate tokens appear only when two bookings arrive in the same second: token allocation was not designed as one indivisible step	Coding. Design chosen because the flaw is in how allocation is structured, not a slip in a statement
I2	Requirements	The case says the Sunday rule was never written in the requirements	None

Incident	Main origin	Evidence from the case	Possible second origin and why I chose the main one
I3	Requirements	The GST rule was misunderstood, so the business rule itself was unclear or wrong	Coding. Requirements chosen because the misreading concerns the business rule, which is requirement knowledge
I4	Coding	A crash on names over 40 characters means length handling is missing	Requirements (no maximum length specified). Coding chosen because a program must not crash on any input whatever the specification says
I5	Coding	The senior rule is simple; only its boundary was coded wrongly	Requirements (the date on which a patient "turns 60" is not stated). Coding chosen because the rule is common knowledge
I6	Design	Two doctors overwrite shared data when saving together; sharing state is a design decision	Coding. Design chosen because data sharing is an architectural choice
I7	Documentation	The manual names a "My Visits" menu that does not exist	None
I8	Testing	The tester compared the result with a wrong expected value	None
I9	Environment / design	The SMS provider changed its API key; the application neither handled nor monitored an external change	Design (no error handling or alerting). Environment is the trigger, because the change came from outside the team, but weak design let it cause harm
I10	Requirements	Nothing in the case shows a performance target	Design or coding (inefficient loading). Requirements chosen because without a target nobody could call 25 seconds a defect

Table 15: Overall pattern of origins

Origin	Incidents	Count	Share of the 10 incidents
Requirements	I2, I3, I10	3	30%
Design	I1, I6	2	20%
Coding	I4, I5	2	20%
Testing	I8	1	10%
Documentation	I7	1	10%
Environment	I9	1	10%

Interpretation. Five of the ten incidents (50%) start in requirements or design, which matches the defect data of the last release (52 of 100 defects introduced in these two phases). Only two incidents (20%) start in coding, yet coding is the only thing the team checked. Three of the ten are not code defects at all (testing, documentation, environment). I1 and I6 share one cause, shared data under simultaneous use, which suggests a design weakness that may exist elsewhere too.

What this means for the hospital. Money should go first into the early phases (requirement

reviews and design inspections) and into an independent test activity, not into more coding skill. These are also the cheapest places to remove defects: ₹500 and ₹1,500 per defect, against ₹40,000 in production.

Task 8 – Evaluate Defect Classes and Severity **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)
- The class, severity and priority of each incident, with reasons
- Why severity and priority may differ for the same defect (for example, I7 and I1)
- Which defect class the team should target first, and why

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

Ans:

Defect Classes, Severity and Priority

I use the four standard classes (requirements/specification, design, coding, testing) with their usual sub-classes. Two incidents fall outside them (the user manual and the external SMS provider), so I add a documentation class and treat the external provider as an external-interface sub-class of design.

Table 16: Defect classification scheme applied to MediQueue

Class	Sub-classes	MediQueue incidents
Requirements / specification	Functional description, feature, business rule, interface description, non-functional (performance) requirement	I2, I3, I10
Design	Algorithm/processing, control logic and sequence, data and shared state, module interface, external interface	I1, I6, I9
Coding	Algorithm/processing, control logic (boundary), data and input handling, initialisation, typographical, code documentation	I4, I5
Testing	Test-case design, test oracle (expected result), test procedure, test harness	I8
Documentation (added)	User manual, help text, release notes	I7

Scale used. Severity is the damage to patients and the hospital: **Critical** = risk to patient safety or privacy, or financial loss to many patients; **High** = a key function is wrong or unusable; **Medium** = a function is wrong but a workaround exists; **Low** = cosmetic or informational. Priority is the order of repair, which combines severity with how often it happens, the workaround, the cost of the fix and the risk of delay: **P1** = fix now, **P2** = next, **P3** = planned.

Table 17: Class, severity and priority of every incident

Incident	Class / sub-class	Severity	Priority	Reason
I1	Design / control logic (concurrency)	High	P3	Two patients sharing a token disrupts the queue, but it needs the same second and the same doctor, and reception can issue a new token, so it follows the urgent items
I2	Requirements / business rule (missing)	Medium	P2	Patients travel to a closed OPD, though safety is not affected; the fix is a cheap rule and many people are affected, so it is done early
I3	Requirements / business rule (wrong)	High	P1	Patients are overcharged on every booking: money and trust are lost and there may be tax or legal consequences
I4	Coding / input handling	High	P3	The app crashes, but only for names over 40 characters; a shorter name is a workaround and the fix is easy
I5	Coding / control logic (boundary)	Medium	P3	Patients turning 60 that day lose their priority; the group is small, but it is a matter of fairness
I6	Design / data and shared state	Critical	P1	One patient's prescription shows another patient's details: a breach of patient safety and privacy
I7	Documentation / user manual	Low	P2	No data is harmed, but every patient following the manual is misled; the fix takes minutes
I8	Testing / test oracle	High	P2	The false "Pass" may hide a refund defect and casts doubt on every other test result
I9	Design / external interface	Medium	P3	Missed reminders cause missed appointments but no harm; needs alerting and key handling
I10	Requirements / non-functional (performance)	Medium	P3	The screen is slow but correct; it needs a target and profiling, so it is planned for the next cycle

Why severity and priority can differ

Severity describes the damage if the defect occurs; priority describes how soon to fix it, which also depends on frequency, workaround, cost and what else is waiting. **I7** is Low in severity but P2 in priority: it harms no data, yet every patient reading the manual is affected and the fix takes minutes. **I1** is High in severity but only P3: the effect is serious, but it needs two bookings in the same second and reception can reissue a token, so with I3 and I6 waiting it comes after them. This pair shows that ranking by severity alone would give the wrong order.

Most harmful class and where to start

The most harmful defects are the design defect on shared data (I6, Critical) and the requirement defect involving money (I3, High, P1). The team's own priorities do not reflect

this: it looked only at visible errors in a demo and had no severity scale, so a crash and a privacy leak would have been treated alike.

I recommend that the team tackles the **requirements/specification class first**, for three reasons: (1) it accounts for 30 of the last release's 100 defects and 3 of the 10 incidents; (2) only 2 of those 30 were caught before release, a detection efficiency of 6.7%; (3) it is the cheapest to fix early (₹500 per defect against ₹40,000 in production). Design inspection of shared data and concurrency should follow immediately, because that class holds the only Critical defect, I6.

Task 9 – Design and Appraise a Defect Repository **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

- The fields a defect repository for MediQueue must contain, and why each is needed
- How the repository supports defect analysis, process improvement and prevention
- The weaknesses of the current e-mail and spreadsheet approach

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

Ans:

A Defect Repository for MediQueue

A repository is the project's memory. For every defect it must answer: what went wrong, where it began, why it was not caught earlier, how serious it was, how it was fixed and how to stop it recurring. Table 18 gives the fields in groups, with the reason for each group.

Table 18: MediQueue defect repository fields and the reason for each

Field group	Fields	Reason it is needed
Identification	Defect ID, title, date reported, reporter, source (patient complaint, tester, review, monitoring)	A unique reference; shows who finds defects and how, so weak detection becomes visible
Description	Description, steps to reproduce, expected result, actual result, attachments	Lets a developer reproduce and fix the defect without questions; separates the failure from expected behaviour
Location	Module/feature, release and build number, environment (production/test), browser or device	Shows where defects cluster and which release holds them

Field group	Fields	Reason it is needed
Classification	Class and sub-class (requirements, design, coding, testing, documentation, environment)	Basis for analysing defect types and building checklists
Origin and detection	Phase introduced, phase detected, detection method (review, unit test, system test, production)	Gives the phase-containment figures of Task 3 and the escape rate
Rating	Severity, priority, patient-safety/privacy flag, number of patients affected	Sets the order of work and highlights serious risks
Handling	Status (new, assigned, fixed, verified, closed, reopened), owner, dates	Tracks each defect to closure and shows ageing
Cause and fix	Root cause (the error), description of fix, effort in hours, cost	Gives cost-per-phase figures and lets the team learn from causes
Traceability	Related requirement ID, test case ID, related or duplicate defects	Shows which requirement or test was missing and links recurring defects
Prevention	Proposed preventive action, owner, status	Turns every defect into a process improvement

How the repository supports analysis, improvement and prevention

- **Defect analysis:** Pareto charts by class, module and phase show where to focus, and counts of origin and detection phase supply the Task 3 figures for every release.
- **Process improvement:** a growing share of requirement defects found late proves the review is weak; many reopened defects point to poor fixing or verification; the data also set targets (for example, fewer than 20% of defects escaping to production).
- **Prevention:** each cause becomes an action (a checklist item, a coding rule, a new regression test). I1 and I6 would add the design-checklist item “shared data under simultaneous use” and a concurrency test in every release.
- **Regression and test design:** past defects supply regression cases and test ideas for similar modules.
- **Management:** trend figures replace “the demo went well” as the basis of release decisions.

Shortcomings of e-mail and spreadsheet logging

Table 19: E-mails and spreadsheet compared with a repository

Aspect	E-mails and spreadsheet	Defect repository
Structure	Free text; people use different words and columns	Fixed fields and lists; everyone uses the same terms
Duplicates	The same problem is reported again and again without being noticed	Duplicates are linked to one defect
Workflow	No status or owner; defects get lost in mailboxes	Status, owner, due dates and reminders
History and audit	No change history; copies of the sheet conflict	Every change logged with date and user

Aspect	E-mails and spreadsheet	Defect repository
Analysis	Counting by class, phase or cost needs manual effort	Built-in reports and filters
Traceability	No link to requirement, test case or build	Linked to all three
Privacy	Patient names and health details travel in e-mails and shared sheets	Access control and masking of patient data

In my judgement the repository pays for itself. A free or low-cost tracking tool will do, filling the fields adds only a few minutes per defect, and in return the hospital gets the evidence it needs to decide where to invest, which it never had before.

Task 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- The root cause of each selected defect
- A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)
- The cost and effort of each strategy compared with its benefit
- Which strategy gives the best value for the hospital

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified, cost-aware prevention strategies.

Ans:

Root Causes and Prevention Strategies for Three Defects

I chose one incident from each of three classes: **I3** (requirements), **I6** (design) and **I5** (coding). For each I give the root cause, the single process change that would have prevented it, and its cost against its benefit.

Table 20: Root cause and prevention strategy of the chosen defects

Incident (class)	Root cause	Prevention strategy (one process change)
I3 – wrong fee charged (requirements)	The fee and tax rule was never written down and agreed with billing; it was implemented twice (display and payment) from different readings	A requirement review with billing staff and a written fee-rule table; the fee is calculated in one place only, and the table serves as the test oracle

Incident (class)	Root cause	Prevention strategy (one process change)
I6 – wrong patient on prescription (design)	The design shares data between users and nobody examined it for simultaneous use; no design review took place	A design inspection with the checklist item "shared data and simultaneous users", plus a concurrency test for saving prescriptions
I5 – no senior priority on the 60th birthday (coding)	The age rule was coded without a boundary check, and nobody else read the code or tested ages 59/60/61	A coding standard for date and age calculations, code review, and unit tests on the boundary values 59, 60 and 61 for every rule

Cost and benefit

Assumptions: a person-hour costs ₹800; a defect that escapes its own phase costs on average ₹32,000 to fix later, being the mean of the testing and production figures in the case, (1,60,000 + 24,00,000) ÷ 80 defects. The strategies are applied in sequence, so overlaps are not counted twice: the requirement review comes first, the design inspection works on what remains, and the coding practices on what remains after both. Reviews are assumed to remove 60% of the defects present in their phase; coding practices lift coding-phase detection from 17.6% to 35%.

Table 21: Cost and benefit of each strategy for one release

Strategy	Effort (hours)	Cost (₹)	Extra defects caught early	Saving per defect (₹)	Total saving (₹)	Benefit ÷ cost
Requirement review	50	40,000	16.0 (18 instead of 2)	31,500 (32,000 – 500)	5,04,000	12.6
Design inspection	75	60,000	18.4 (20.4 instead of 2.0)	30,500 (32,000 – 1,500)	5,60,000	9.3
Coding standard, code review, boundary-value unit tests	80	64,000	9.0 (18.1 instead of 9.1)	29,000 (32,000 – 3,000)	2,59,700	4.1
All three together	**205**	**1,64,000**	**43.3**	**–**	**13,23,700**	**8.1**

All three strategies pay for themselves: each returns at least four times its cost, and together they return about eight times. Values are rounded to the nearest ₹100.

Best value: the requirement review. It has the highest ratio (12.6), needs only a few hours of discussion with billing and OPD staff, needs no tools, and targets the largest group of defects (30 of 100). It would have prevented I3, I2 and I10, and it supports the other strategies, because test oracles are drawn from reviewed requirements. The design inspection has a lower ratio (9.3), but I would still keep it because it deals with the Critical defect I6, whose real cost (a privacy breach) is far above the ₹40,000 used in the model.

Task 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for

MediQueue.

Discuss:

- The process changes you recommend and their order of priority
- The role of testers, reviews and the defect repository in your plan
- The expected effect on the number and cost of defects
- Trade-offs, risks and limitations of your plan
- Whether the plan remains practical for a small team of five developers

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected

A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

Ans:

Quality Improvement Plan for MediQueue

The plan rests on the evidence of Tasks 1–10: half the incidents and 52% of the defects begin in requirements and design; only 9.6% of those are caught early; production defects carry 91.9% of the cost; and there is no tester, no exit criterion and no repository. Changes are ranked by value for money and by what five developers can realistically absorb.

Table 22: Quality improvement plan in order of priority

Priority	Change	Role of tester / reviews / repository	Addresses	Effort
1	Release gate with written exit criteria (no open Critical or High defect, planned high-risk tests done, at least 95% passed), signed off by the hospital	Tester collects the evidence; management decides	Demo-based approval; all incidents	A few hours per release
2	Review of every rule (fees, working days, age rules, performance target) with billing and OPD staff; keep a business-rule table	Tester and developers review; the table is the test oracle	I2, I3, I5, I10	About 50 person-hours
3	Appoint an independent tester (full-time, or a trained hospital IT staff member)	Writes the test plan, designs tests by equivalence classes and boundaries, runs system tests, reviews test cases (I8)	I4, I5, I7, I8, I10	One person for 8 months
4	Design inspection of high-risk areas (shared data, concurrency, external interfaces); add concurrency and load tests	Tester and two developers; checklist held in the repository	I1, I6, I9	About 75 person-hours
5	Defect repository with the Task 9 fields; monthly review of defects by class, phase and cost	Whole team; the tester owns the reports	Learning from every incident	1 day to set up, 5 minutes per defect
6	Coding standard, peer review and boundary-value unit tests by developers	Developers	I4, I5	About 80 person-hours

Priority	Change	Role of tester / reviews / repository	Addresses	Effort
7	Document review (walk through the manual on the live app); interface monitoring and alerting for the SMS provider	Tester; operations	I7, I9	A few hours

Expected effect on the number and cost of defects

I applied the Task 3 defect-flow model with three assumptions: requirement and design reviews remove 60% of defects present; coding-phase detection rises from 17.6% to 35% (code review and unit tests); testing-phase detection rises from 26.7% to 60% (planned tests and an independent tester). Plan costs: tester ₹50,000 a month × 8 months = ₹4,00,000; reviews ₹1,00,000 (125 hours × ₹800); code review and unit tests ₹64,000 (80 hours × ₹800); training and tools ₹50,000.

Table 23: Expected outcome for one release of the same size as the last

Measure	Last release	With the plan
Defects found before release	40	79.6
Defects reaching patients	60	20.4
Cost of fixing defects (₹)	26,10,500	10,95,412
Cost of the plan (₹)	–	6,14,000
Total cost (₹)	26,10,500	17,09,412
Net saving (₹)	–	9,01,088 (34.5%)

Defects reaching patients fall by about two thirds (60 to about 20), and the plan saves around ₹9 lakh per release after paying for itself. Benefits that cannot be priced, such as fewer patient-safety and privacy incidents, come on top.

Trade-offs, risks and limitations

- **Slower first release:** reviews and testing take calendar time, and the hospital must accept a gate that may delay a release.
- **Assumed efficiencies:** the 60%, 35% and 60% figures are estimates; if reviews are poor the saving will be smaller. I would record real figures in the repository and update them.
- **Reviews as ritual:** reviews can decay into box-ticking; checklists and recorded findings keep them meaningful.
- **Tester–developer friction:** the tester must be seen as part of the team and report defects without blame.
- **Uncounted costs:** patient harm and loss of trust are left out of the figures, so the true benefit is larger.
- **Single-release data:** the estimate rests on one release of 100 defects.

Is it workable for five developers?

Yes, if it stays lightweight. Developer effort is about 125 hours for the two reviews plus 80 hours for code review and unit tests, 205 hours in all. Five developers over eight months give 5 × 8 months × 20 days × 8 hours = 6,400 hours, so the plan takes about 3.2% of developer time, plus one tester. Reviews are short (90 minutes), checklist-driven and aimed at the highest-risk areas, and the repository tool can be free. The one large cost is the tester, who is also the largest source of savings.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Quality Process **8 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- What has changed from the original situation
- Which assumptions in your quality improvement plan are affected
- Why frequent releases change the role of testing and the tester
- Why sharing health records raises the severity of some defect classes

Evidence Expected

A clear analysis of how the changed conditions affect the original quality improvement plan.

Ans:

Impact of the New Conditions

What has changed

Table 24: Original situation compared with the changed situation

Aspect	Original situation	Changed situation
Release rhythm	One release roughly every six to eight months	A release every two weeks (26 a year, 13 times as often)
Interfaces	The team's own application and an SMS provider	Also the national digital health ID system run by an external agency
Data	Appointment and prescription data of a single hospital	Health records shared across hospitals
Control	The team controls all its code and release timing	The agency decides when and how the ID system changes

Assumptions in my plan that are affected

- **One large review per release:** the plan assumes whole-document reviews of about 125 person-hours. That cannot be repeated every fortnight; reviews must be small and tied to each change.
- **Manual testing by one tester:** a full manual regression run (say 400 cases × 6 minutes = 40 person-hours) would eat half of the 80 hours one tester has in a two-week cycle.
- **A test bed we control:** with an external ID system, the test environment is the agency's sandbox, which may not behave like the live service.
- **A repository for one system:** it assumes every defect lies in our code, but some will now lie in the agency's system or in the contract between the two.
- **₹40,000 per production defect:** this rises when a defect can pass wrong data to other hospitals.

Why frequent releases change the role of testing and the tester

With a release every two weeks there is no "testing phase at the end". Testing must run alongside development, and a tester who joins only at the end would become a permanent bottleneck. The tester joins from story refinement onward, spends less time on repetitive checks (which are automated and run on every change) and more on risk analysis, exploratory testing, test automation and the release decision. Every release must also be reversible and monitored, because a faulty release reaches 6,000 patients a day.

Why shared health records raise severity

Once records are shared, an I6-type defect (one patient's data in another's document) is no longer a local error but a confidentiality breach across hospitals, with legal duties and possible harm to a patient treated on wrong information. Design and data defects (shared state, identity matching) and interface defects therefore become Critical, and so would an access-control defect that was Medium before. Requirement defects about consent and identity also become Critical, while defects like I7 (the manual) stay Low.

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- Whether the planned reviews and test activities fit into a two-week cycle
- Which new defect origins are introduced by the external health ID system
- Whether your defect repository is still sufficient

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

Ans:

Is the Original Plan Still Suitable?

Table 25: Fit of the planned activities to a two-week cycle

Planned activity	Fits two weeks?	Reason and change needed
Review of all rules (about 50 person-hours)	No, not as it is	Review only the rules in that cycle's stories (30–45 minutes per story) and keep the rule table up to date
Design inspection (about 75 person-hours)	Partly	Inspect only stories involving shared data, identity or external interfaces; keep the checklist
Independent tester doing manual testing	No	Need about 114 + 40 (regression) + 30 (ID system and privacy checks) = 184 person-hours per cycle against about 140 available (tester 80 + 15% of the developers' 400 = 60). That is 31% over capacity
Release gate	Yes	Keep; automate the checks that feed it
Code review and unit tests	Yes	Keep, as part of the definition of done
Defect repository	Partly	Needs new fields (below)

The capacity figures are my estimates. The original plan needs about 1,485 person-hours (a tester for 8 months, 8 × 20 × 8 = 1,280 hours, plus 205 developer hours). Releases are now 13 times as frequent (26 against 2 a year), so I divide by 13 to get about 114 hours per release, then add one full manual regression run (40 hours) and the new ID-system and privacy checks (30 hours).

New defect origins brought by the external health ID system

- **Contract and version defects:** the agency changes a field, version or rule without telling us (the same pattern as I9).
- **Data-mapping and identity defects:** patients matched wrongly, inconsistent name and date formats, duplicate IDs.
- **Availability and speed:** the external service is slow or down, and our app freezes or loses the booking.
- **Authentication and consent defects:** expired tokens, missing patient consent, wrong

access rights.

- **Environment differences:** the agency's test environment behaves differently from the live service.
- **Privacy and security defects:** records exposed in transit or in logs.

Is the defect repository still sufficient?

No. It was designed for defects in the team's own system. It needs extra fields: whether the defect is internal, in the agency's system or in the interface; the interface version or contract reference; the agency's ticket number and response date; data sensitivity and the number of patients or hospitals affected; whether it must be reported to the authority; the release and pipeline stage where it was found; and the time taken to detect it and to restore service. Without these the team cannot tell its own defects from the agency's or see how quickly problems are found and fixed.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

Discuss:

- Describe the revised process and the tester's revised role
- Explain which testing principles now become more important
- Explain how defects from the external system will be classified and tracked

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear quality-engineering reasoning.

Ans:

Decision: MODIFY the Quality Process

I decide to **modify** the plan. I do not retain it unchanged, because Table 25 shows its reviews and manual testing cannot fit a two-week cycle (about 31% over capacity) and it has no way to handle an external system. I do not redesign it from nothing, because the evidence it rests on still holds: over half the defects begin in requirements and design, early detection is far cheaper (₹500 or ₹1,500 against ₹40,000), and an independent tester, a release gate and a repository are still needed. What changes is how often and how small each activity is, and what is automated.

The revised process

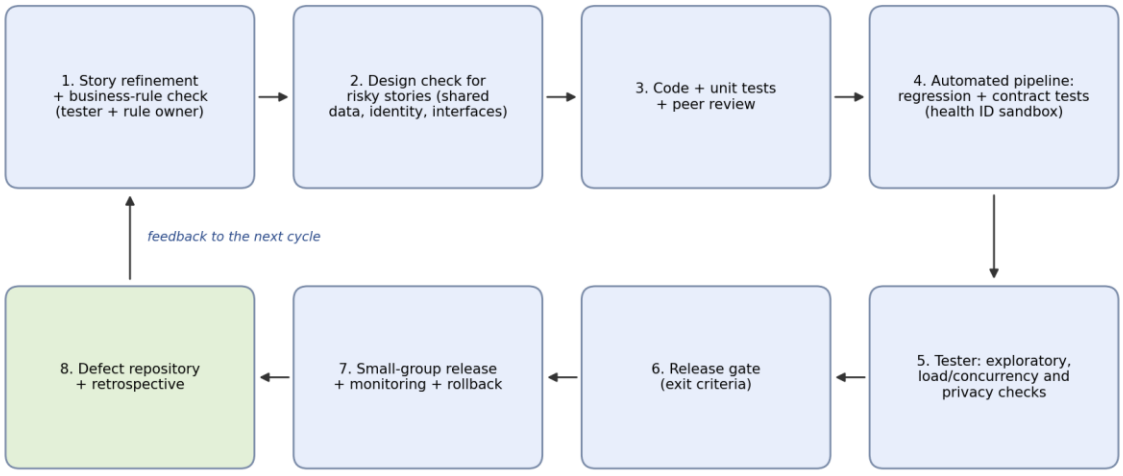


Figure 2: Revised quality cycle of two weeks

- **Story refinement:** every story gets acceptance criteria and a business-rule check by the tester and the hospital owner of that rule (30–45 minutes).
- **Design check:** only for high-risk stories (shared data, identity, external interface), using the checklist.
- **Development:** peer review and boundary-value unit tests are part of the definition of done.
- **Automated pipeline:** every change triggers automated regression tests on the critical paths (booking, token, fee, prescription, reminder) and contract tests against the health ID sandbox. The 40-hour manual regression run becomes an automated run of a few minutes.
- **Tester checks:** exploratory testing of the changed stories, concurrency and load tests in the nightly run, and privacy checks for any change touching health records.
- **Release gate and safe rollout:** exit criteria are checked from the pipeline results; the release goes first to a small group and can be rolled back; the SMS and health ID interfaces are monitored with alerts.
- **Retrospective:** a defect repository review every cycle to tune checklists and tests.

The tester's revised role

The tester becomes a quality coach and test engineer inside the team: analysing the risk of each story, building and maintaining the automated regression and contract tests, doing exploratory and privacy testing, owning the release-gate evidence and repository reports, and acting as the contact with the agency for interface problems. The tester is no longer only a final checker.

Principles that gain importance

- **Early testing and planning:** in two weeks there is no time for late correction.
- **Repeatable, reusable tests (regression):** every release must re-prove existing behaviour, so automation is essential.
- **Testing depends on context:** health data and money call for stricter testing than a

general-purpose app.

- **Defect clustering:** the interface and shared-data areas are where most new defects will appear.
- **Test invalid and unexpected inputs, especially at interfaces:** wrong, late or missing replies from the agency are normal cases to test.

Classifying and tracking defects from the external system

Table 26: Additional classification for external-system defects

Item	How it is handled
Class	A new "External interface" class with sub-classes: contract/version, data mapping, availability/performance, authentication/consent, environment difference
Owner	Internal (our adapter), agency (their service) or joint (contract unclear); this decides who fixes it
Severity	Judged by impact on patients and data: any wrong record or privacy exposure is Critical
Tracking	Our defect ID is linked to the agency's ticket; response times are logged; any workaround (queue, retry, manual process) is recorded
Measures	Time to detect, time to restore, number of such defects per release, share caused by agency changes
Prevention	Contract tests on every build; production monitoring; a named contact and a shared release calendar with the agency

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Discuss:

- The defect data and cost figures
- Defect origins and classes
- Release frequency
- Patient-safety and data-privacy risk
- Team size and capacity

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

Ans:

Defending the Final Decision

Table 27: Evidence supporting the decision to modify the plan

Evidence	What it shows	Effect on my decision
Defect data and cost	Patients found 60% of the defects, and those defects caused 91.9% of the cost (₹24,00,000 of ₹26,10,500); a production defect costs 80 times a requirements defect	Early detection still pays, so I keep the plan's core
Origins and classes	50% of incidents and 52% of defects arise in requirements and design; coding is only 20% of incidents	Keep requirement and design checks, but small and per story
Release frequency	26 releases a year; the original plan needs about 184 person-hours per cycle against about 140 available (31% over)	Unchanged, the plan would fail; smaller reviews and automation are needed
Patient safety and privacy	Shared records turn I6-type defects into breaches across hospitals (Critical)	Add design checks, privacy testing and a strict release gate
External agency	A new defect origin outside our control (contract, availability, mapping)	Add contract tests, monitoring and a new repository class
Team size and capacity	Five developers (about 6,400 hours over eight months) plus one tester	The changes use about 3% of developer time plus the tester; a full redesign with a larger test team is unaffordable

Why not retain. Unchanged, the manual review and test effort would exceed capacity by 31%, and the team would either drop activities (going back to the old situation in which 60 defects escaped) or delay every release.

Why not redesign. Redesigning everything would throw away the parts the case data strongly support, at a cost five developers cannot carry. The modified plan keeps the proven elements, shrinks them to fit two weeks and automates the repeated checks.

Expected benefit. With the effect assumed in Task 11 (about 20 defects reaching patients instead of 60 for the same amount of work) the saving was about ₹9 lakh per old-sized release. The same improvement is reachable at the new pace, and the release gate, monitoring and rollback limit the damage of any defect that still escapes to two weeks of exposure instead of six months.

Risks I accept, and how I will watch them. (1) The automated regression suite takes time to build, so early cycles will have lower coverage; the tester starts with the critical paths. (2) The review and test efficiencies are estimates; the repository will show the real figures after about three cycles. (3) The agency's behaviour is outside the team's control; contract tests and monitoring give early warning. **Review trigger:** if the repository shows that agency interface defects dominate and the team cannot control them, I would reconsider the decision and redesign the process around a dedicated integration test group and a formal agreement with the agency.

MARK DISTRIBUTION

Task	Assessed area	Marks
Task 1	Evaluate Testing as an Engineering Activity	5
Task 2	Evaluate the Role of Process in Software Quality	2
Task 3	Evaluate Testing as a Process	6
Task 4	Apply and Critique Basic Definitions	4
Task 5	Evaluate the Application of Testing Principles	3
Task 6	Evaluate the Tester's Role	3
Task 7	Trace the Origins of Defects	4
Task 8	Evaluate Defect Classes and Severity	4
Task 9	Design and Appraise a Defect Repository	3
Task 10	Evaluate Defect Examples and Prevention Strategies	4
Task 11	Recommend and Defend a Quality Improvement Plan	4
Task 12	Day-4 Design Challenge (A–D, 2 marks each)	8
Total		50